

Documentazione Progetto SAD

Gruppo D6

Anno 2025/2026

Contents

1	Identificazione del Team e Obiettivi	1
1.1	Descrizione Task	1
1.2	Descrizione Approccio di Sviluppo	1
2	Analisi dei Requisiti	5
2.1	Requisiti Funzionali	5
2.2	Requisiti Non Funzionali	5
2.3	Analisi dell'Impatto	6
2.3.1	Migrazione Database	6
2.3.2	Gestione Suggerimenti	19
2.4	Diagramma dei Casi d'Uso	20
2.5	Scenari	22
3	Progettazione della Soluzione	25
3.1	Decisioni di Progetto	25
3.2	Viste Architetture	27
3.2.1	Component Diagram	27
3.3	Dettagli Tecnici di Implementazione	28
3.3.1	Package Diagram	28
3.3.2	Sequence Diagram	29
3.3.3	Deploy Diagram	36
3.3.4	Nuove Interfacce REST	38
4	Implementazione	41
4.1	Tabella Moduli Aggiunti e Modificati	41
5	Test	43
6	Sviluppi Futuri	46

1 Identificazione del Team e Obiettivi

- **ID Team:** D6
- **Task Assegnato:** TaskR2 - Caricamento Suggerimenti e Migrazione MongoDB
- **Componenti Gruppo:**

Nome Cognome	Email Istituzionale
Sorrentino Simone	simone.sorrentino6@studenti.unina.it
Antonio Varese	an.varese@studenti.unina.it
Rosa Palmentieri	ros.palmentieri@studenti.unina.it
Fabio Chiacchio	fab.chiacchio@studenti.unina.it

- **URL Versione di Partenza:**
<https://github.com/Testing-Game-SAD-2023/A13>
- **URL Repository Consegnato:**
https://github.com/Testing-Game-SAD-2023/A13/tree/D6_TaskR2_CaricamentoSuggerimenti_MigrazioneMongoDB

1.1 Descrizione Task

La Task R2 è relativa al microservizio T1 e si compone di due obiettivi principali:

1. Caricamento e gestione di suggerimenti di carattere generico e associati a Classi Under Test.
2. Migrazione del database da MongoDB verso un database SQL, mantenendo la compatibilità con le API esistenti.

1.2 Descrizione Approccio di Sviluppo

L'approccio di sviluppo adottato è di tipo iterativo e caratterizzato dalle seguenti fasi:

1. Analisi dei requisiti
2. Progettazione della soluzione
3. Implementazione della soluzione
4. Testing
5. Deploy della soluzione

In particolare, per la **migrazione del database** si è reso necessario un ulteriore step di comprensione dello stato di partenza del database NoSQL MongoDB, in modo da poter poi progettare correttamente la soluzione di migrazione.

Per poter individuare lo stato di partenza del database MongoDB, è stato utilizzato a supporto il tool **MongoDB Compass**. Si tratta di un'interfaccia grafica per MongoDB, che permette di esplorare, interrogare, analizzare e gestire i dati NoSQL.

Pertanto, l'applicazione è stata avviata apportando una modifica al *docker-compose.yml* del microservizio T1, per esporre la porta del servizio *mongo_db* e connettere MongoDB Compass al database.

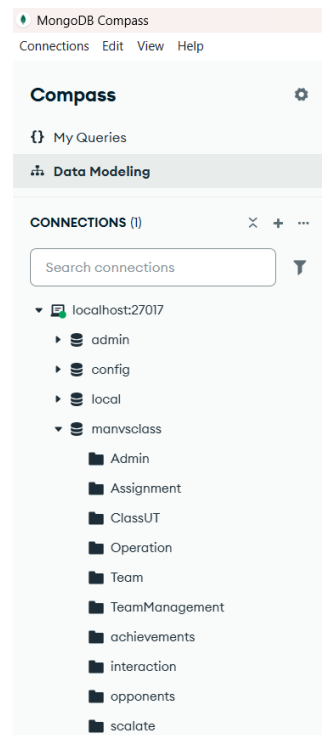
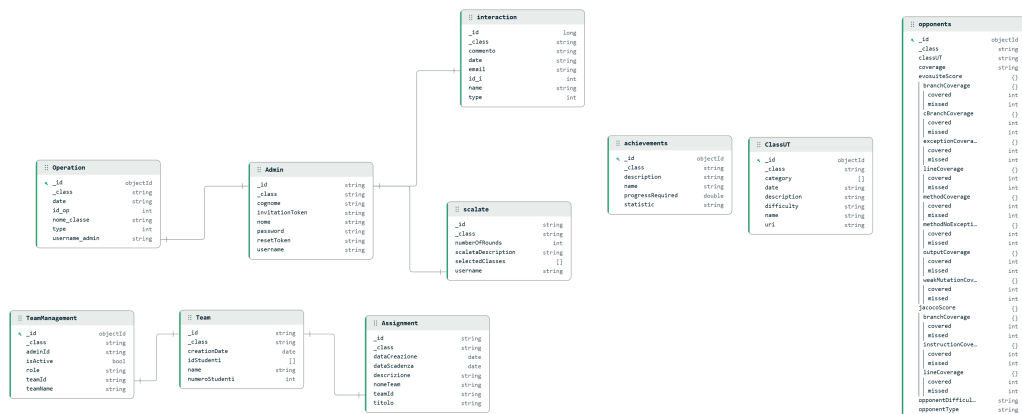


Figure 1: Connessione MongoDB Compass al database

MongoDB Compass consente di definire un modello dei dati a partire da un database esistente. Nello specifico, quello che viene generato è un **Diagramma dello Schema dei Documenti**, che mostra la struttura interna (schema) dei documenti presenti in ogni collezione. Si tratta di un diagramma che viene generato automaticamente a partire da un'analisi campionaria dei documenti presenti nel database. Quindi, mostra la struttura interna dei JSON per ogni



Un'osservazione importante da fare è che Compass prova ad interpretare le relazioni tra le collections basandosi sui dati che ha analizzato. Questo, è di grande aiuto nel processo di migrazione verso un database SQL. Infatti, in un database NoSQL come MongoDB non esistono i vincoli di integrità referenziale, pertanto alcuni legami mostrati nel diagramma tramite delle linee non sono da intendersi come delle Foreign Key, ma sono piuttosto delle **relazioni logiche**, dedotte sulla base dei documenti presenti nelle collections.

Sulla base del Diagramma dello Schema dei Documenti mostrato possiamo classificare i legami tra le collezioni in due categorie principali:

- Admin - Scalate
- Admin - Interaction
- Admin - TeamAdmin
- Team - Assignment
- Team - TeamManagement

2. **Relazioni basate su Stringhe:** si tratta di relazioni implicite, che non appaiono come linee nel diagramma di Compass perché utilizzano il nome (stringa) anziché l'ID tecnico. In particolare, ci sono:

- **ClassUT - Opponent**

- il campo *classUT* di Opponent si riferisce al campo *name* di ClassUT

- **ClassUT - Interaction**

- il campo *name* di Interaction si riferisce al campo *name* di ClassUT

Questa analisi delle dipendenze logiche ha rappresentato la base per il processo di migrazione verso un database relazionale, permettendo di trasformare un modello flessibile ma privo di vincoli (MongoDB) in uno schema ER coerente e robusto (PostgreSQL), garantendo l'integrità referenziale richiesta dai requisiti non funzionali.

2 Analisi dei Requisiti

2.1 Requisiti Funzionali

- RF1 Migrazione Database:** Il sistema deve migrare l'attuale database del microservizio T1 da MongoDB verso un database relazionale (SQL) senza modificare il comportamento preesistente del servizio.
- RF2 Caricamento Suggerimenti** Il sistema deve consentire all'amministratore di caricare dei suggerimenti generici oppure associati ad una specifica classe da testare tramite un file JSON. Devono essere effettuati dei controlli su formato, struttura e contenuto del file. I suggerimenti devono essere caratterizzati dalle seguenti informazioni: nome, contenuto, tipo, ordine di visualizzazione, email dell'amministratore che crea il suggerimento, eventuale nome della classe UT di riferimento ed eventuale immagine allegata.
- RF3 Visualizzazione Suggerimenti** Il sistema deve consentire all'amministratore e al microservizio T5 di visualizzare l'elenco dei suggerimenti ed il relativo dettaglio. Inoltre, il sistema deve consentire il recupero dei suggerimenti filtrandoli in base alla tipologia, distinguendo tra suggerimenti generici e suggerimenti associati a una specifica Classe Under Test.
- RF4 Modifica Suggerimenti** Il sistema deve consentire all'amministratore di modificare nome, contenuto e immagine del suggerimento.
- RF5 Eliminazione Suggerimenti** Il sistema deve consentire all'amministratore di eliminare i suggerimenti.
- RF7 Ordinamento Suggerimenti** Il sistema deve consentire all'amministratore di indicare l'ordine con cui i suggerimenti dovranno essere visualizzato da parte del servizio T5.

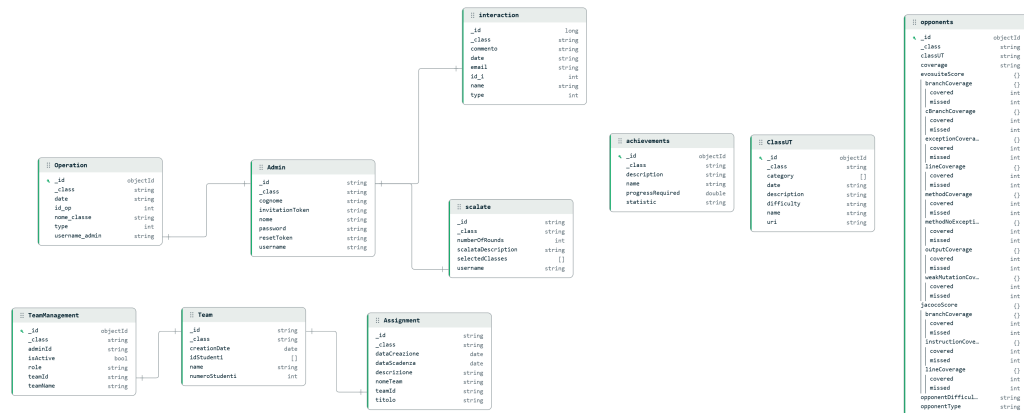
2.2 Requisiti Non Funzionali

- RNF1 Manutenibilità** La soluzione deve favorire la manutenibilità del sistema, garantendo una chiara separazione tra livello di persistenza, logica applicativa e interfacce, e adottando uno schema di database relazionale chiaro.
- RNF2 Integrità e affidabilità dei dati** Il sistema deve garantire l'integrità e l'affidabilità dei dati, mediante l'utilizzo di vincoli di database quali chiavi primarie, chiavi esterne e vincoli di integrità.
- RNF3 Compatibilità architetturale** La soluzione deve essere compatibile con l'architettura complessiva del sistema, mantenendo la coerenza delle API esistenti e garantendo la corretta integrazione con gli altri servizi del sistema.

RNF6 Sicurezza e controllo degli accessi Il sistema deve garantire il controllo degli accessi alle funzionalità di gestione dei suggerimenti, consentendo le operazioni di creazione, visualizzazione, aggiornamento ed eliminazione dei suggerimenti esclusivamente agli utenti con ruolo di amministratore.

Analizziamo separatamente l'impatto relativo alla migrazione del database e quello relativo alla gestione dei suggerimenti nelle seguenti sottosezioni.

Per comprendere l'impatto effettivo sul microservizio, riportiamo per comodità di visualizzazione il Diagramma dello Schema dei Documenti MongoDB mostrato in precedenza.



6

Di seguito riportiamo invece il **Diagramma ER** del database relazionale corrispondente. Tale diagramma mostra come le informazioni saranno archiviate sul nuovo database ed in che modo sono in relazione tra loro. Pertanto, l'obiettivo è mostrare:

- come abbiamo trasformato le collezioni NoSQL in tabelle SQL;
- che tipi di dati sono stati adottati;
- in che modo è garantita l'integrità dei dati mediante i vincoli di Foreign Key.

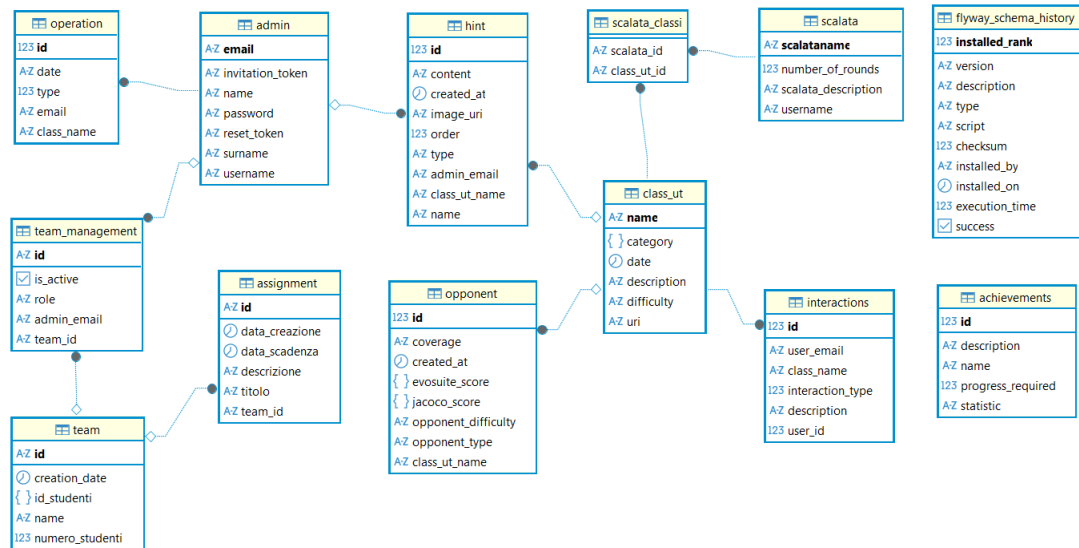


Figure 4: Diagramma ER PostgreSQL

Procediamo con un'analisi del mapping delle collezioni NoSQL verso le tabelle SQL.

Table 1: Mappatura Collezione **Opponents** → Tabella **Opponent**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Int, PK)	Passaggio da identificativo alfanumerico a numerico autoincrementale.
classUT (String)	class_ut_name (FK)	La stringa libera diventa una Foreign Key vincolata all'integrità referenziale verso la tabella class_ut.
coverage (String)	coverage (Varchar)	Mappatura diretta del valore di copertura complessiva.
opponentType (String)	opponent_type (Varchar)	Conversione del tipo di sfidante (es. Robot, User) in formato stringa SQL.
opponentDifficulty (String)	opponent_difficulty (Varchar)	Mappatura della difficoltà associata allo sfidante.
evosuiteScore (Object)	evosuite_score (JSONB)	Mantenimento della struttura complessa nidificata tramite tipo nativo JSONB.
jacocoScore (Object)	jacoco_score (JSONB)	Preservazione dei dettagli di copertura (branch, line, ecc.) in formato JSONB.

Table 2: Mappatura Collezione **ClassUT** → Tabella **Class_ut**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
name (String)	name (PK)	Il nome della classe rimane l'identificativo naturale e la Chiave Primaria.
date (String)	created_at (Timestamp)	Conversione della stringa data nel tipo temporale nativo di PostgreSQL.
difficulty (String)	difficulty (Varchar)	Mappatura diretta della stringa indicante il livello di difficoltà.
description (String)	description (Text)	Trasferimento della descrizione testuale della classe.
uri (String)	uri (Varchar)	Percorso del file nel filesystem (VolumeT0), mantenuto come riferimento.
category (Array)	category (JSONB)	L'array di stringhe (tag/-categorie) viene mappato nel formato JSONB per preservare la flessibilità originale.

Table 3: Mappatura Collezione **Admin** → Tabella **Admin**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
email (String)	email (Varchar, PK)	Mantenuto come identificativo univoco per l'autenticazione.
nome (String)	name (Varchar)	Mappatura del nome dell'amministratore.
cognome (String)	surname (Varchar)	Mappatura del cognome dell'amministratore.
password (String)	password (Varchar)	Trasferimento della stringa della password.
username (String)	username (Varchar)	Mappatura del nome utente scelto dall'admin.
invitationToken (String)	invitation_token (Varchar)	Gestione dei token di invito per la registrazione di nuovi admin.
resetToken (String)	reset_token (Varchar)	Gestione del token per il recupero della password.

Table 4: Mappatura Collezione **Scalata** → Tabella **Scalata**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	scalataname (PK)	In MongoDB l'ID era un ObjectId, mentre nello schema SQL il nome della scalata (<i>scalataname</i>) è stato elevato a Chiave Primaria.
class_ut_name (Inferred)	class_ut_name (FK)	Relazione logica in NoSQL trasformata in un vincolo fisico verso la tabella <i>class_ut</i> .
numberOfRounds (Int)	number_of_rounds (Int)	Mappatura diretta del numero di round previsti per la scalata.
scalataDescription (String)	scalata_description (Text)	Trasferimento della descrizione testuale della scalata.
username (String)	username (FK)	Riferimento all'utente che ha creato la scalata, ora vincolato alla tabella <i>admin</i> .
selectedClasses (Array)	Tabella <i>scalata_classi</i>	L'array NoSQL è stato normalizzato in una tabella di associazione per gestire la relazione Many-to-Many con le classi UT.

Table 5: Mappatura Collezione **Interaction** → Tabella **Interactions**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Int, PK)	L'identificativo alfanumerico di MongoDB viene convertito in una chiave primaria numerica autoincrementale.
email (String)	user_email (FK)	Trasformato in una Foreign Key verso la tabella <i>admin</i> per garantire l'integrità referenziale dell'autore.
name (String)	class_name (FK)	Il riferimento testuale alla classe diventa una Foreign Key verso la tabella <i>class_ut</i> .
commento (String)	description (Text)	Mappatura del contenuto testuale dell'interazione o del feedback lasciato dall'utente.
date (String)	created_at (Timestamp)	Conversione della stringa temporale nel tipo nativo Timestamp per analisi cronologiche precise.
type (Int)	interaction_type (Int)	Mappatura del codice numerico che identifica la tipologia di interazione.

Table 6: Mappatura Collezione **Achievements** → Tabella **Achievements**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Int, PK)	L'identificativo alfanumerico viene convertito in una chiave primaria numerica autoincrementale.
name (String)	name (Varchar)	Mappatura del nome univoco dell'obiettivo o traguardo.
description (String)	description (Text)	Trasferimento della descrizione estesa che spiega le condizioni per sbloccare l'achievement.
progressRequired (Double)	progress_required (Double)	Valore numerico che indica la soglia necessaria per il completamento del traguardo.
statistic (String)	statistic (Varchar)	Nome della metrica o statistica di riferimento utilizzata per il calcolo del progresso.

Table 7: Mappatura Collezione **Operation** → Tabella **Operation**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Int, PK)	L'identificativo tecnico di MongoDB viene convertito in una chiave primaria numerica autoincrementale.
username_admin (String)	email (FK)	Mappatura del riferimento all'amministratore, trasformato in una Foreign Key verso la tabella <i>admin</i> .
nome_classe (String)	class_name (FK)	Il riferimento testuale alla classe diventa una Foreign Key vincolata verso la tabella <i>class_ut</i> .
id_op (Int)	type (Int)	Mappatura del codice identificativo dell'operazione per mantenere la compatibilità con la logica applicativa.
date (String)	date (Timestamp)	Conversione della stringa data nel tipo temporale nativo PostgreSQL per facilitare l'ordinamento cronologico.

Table 8: Mappatura Collezione **Team** → Tabella **Team** (Corretta)

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Varchar, PK)	L'identificativo di MongoDB viene mantenuto come stringa (Varchar) per garantire la coerenza tra i sistemi durante la migrazione.
name (String)	name (Varchar)	Mappatura diretta del nome del team.
creationDate (Date)	creation_date (Timestamp)	Conversione dell'oggetto Date di MongoDB nel tipo nativo Timestamp di PostgreSQL.
idStudenti (Array)	id_studenti (JSONB)	Invece della normalizzazione classica, si utilizza il tipo nativo JSONB di PostgreSQL per memorizzare la lista di ID studenti, mantenendo la struttura a documenti originale per performance di lettura.
numeroStudenti (Int)	numero_studenti (Int)	Mappatura del conteggio dei membri appartenenti al team.

Table 9: Mappatura Collezione **TeamManagement** → Tabella **team_management** (Revisionata)

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (String)	id (Varchar, PK)	L'identificativo univoco della relazione viene mantenuto come chiave primaria per preservare i riferimenti originali.
adminId (String)	admin_email (FK)	Il riferimento all'amministratore è mappato come Foreign Key verso la colonna <i>email</i> della tabella <i>admin</i> .
teamId (String)	team_id (FK)	Il riferimento al team è mappato come Foreign Key verso la colonna <i>id</i> della tabella <i>team</i> .
teamName (String)	-	Campo rimosso; l'informazione è ora recuperabile tramite la relazione con la tabella <i>team</i> (Normalizzazione).
role (String)	role (Varchar)	Mappatura diretta del ruolo dell'utente nel team.
isActive (Boolean)	is_active (Boolean)	Mappatura del flag di stato della relazione.

Table 10: Mappatura Collezione **Assignment** → Tabella **Assignment**

Campo MongoDB	Campo PostgreSQL	Note di Migrazione
_id (ObjectId)	id (Int, PK)	L'identificativo tecnico di MongoDB viene convertito in una chiave primaria numerica autoincrementale.
titolo (String)	titolo (Varchar)	Mappatura del titolo descrittivo dell'assegnazione.
descrizione (String)	descrizione (Text)	Trasferimento del contenuto testuale che descrive i dettagli del compito.
dataCreazione (Date)	data_creazione (Timestamp)	Conversione dell'oggetto Date di MongoDB nel tipo nativo Timestamp di PostgreSQL.
dataScadenza (Date)	data_scadenza (Timestamp)	Mappatura della data di scadenza per il completamento dell'assegnazione.
teamId (String)	team_id (FK)	Il riferimento testuale al team è stato normalizzato come Foreign Key verso la tabella <i>team</i> .
nomeTeam (String)	-	Campo rimosso per evitare ridondanza; il nome del team è ora recuperabile tramite JOIN con la tabella <i>team</i> .

Di seguito è riportata una tabella che raccoglie tutti i vincoli d'integrità referenziale aggiunti nel modello SQL.

Table 11: Principali vincoli di integrità referenziale introdotti nel database PostgreSQL.

Vincolo (Foreign Key)	Motivazione Tecnica e Benefici
<i>opponent</i> → <i>class_ut</i>	Garantisce che ogni sfidante (Robot) sia associato a una Classe Under Test esistente. Impedisce la creazione di record "orfani" in caso di eliminazione di una classe.
<i>interactions</i> → <i>admin</i>	Assicura che ogni interazione o report sia riconducibile a un amministratore regolarmente registrato, validando l'autore a livello di schema.
<i>scalata_classi</i> → <i>scalata</i> e <i>class_ut</i>	Gestisce la relazione Many-to-Many normalizzando l'array NoSQL <i>selectedClasses</i> . Il vincolo assicura che una scalata referenzi solo classi presenti nel catalogo.
<i>hint</i> → <i>class_ut</i>	Vincola i suggerimenti specifici a una classe esistente, permettendo la cancellazione a cascata dei suggerimenti se la classe viene rimossa.
<i>hint</i> → <i>admin</i>	Assicura la tracciabilità di chi ha caricato o modificato un suggerimento, collegando l'entità <i>Hint</i> all'email dell'amministratore.
<i>assignment</i> → <i>team</i>	Formalizza il legame tra compito e gruppo di studenti. A differenza di MongoDB, non è possibile assegnare task a Team ID inesistenti o digitati erroneamente.
<i>team_management</i> → <i>admin</i>	Assicura che un utente associato a un team come supervisore o membro esista effettivamente nella tabella <i>admin</i> . Impedisce di assegnare la gestione di un team a email inesistenti o rimosse dal sistema.
<i>team_management</i> → <i>team</i>	Collega fisicamente l'associazione al team di riferimento. Questo vincolo è cruciale per la coerenza: se un team viene eliminato, l'associazione viene rimossa automaticamente, evitando residui logici nel database.

Per comprendere meglio l'impatto che la migrazione ha sul microservizio T1, riportiamo di seguito il **Diagramma delle classi**, che rappresenta la struttura statica del microservizio e mostra in che modo le tabelle del database si traducono in classi Java.

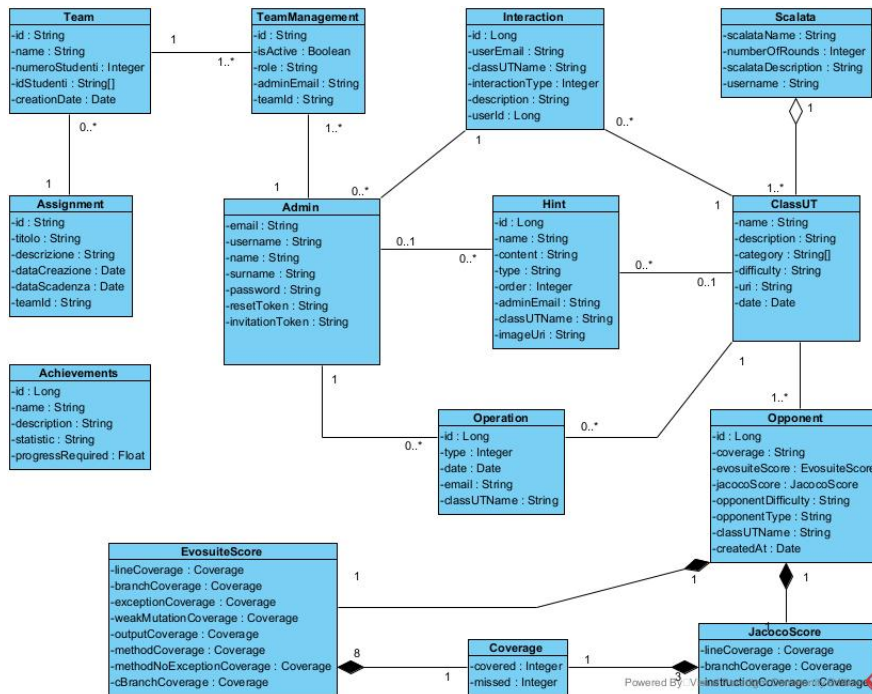


Figure 5: Diagramma delle Classi

In questo diagramma:

- **Admin:** Rappresenta l'utente amministratore del sistema, centrale nella gestione delle operazioni, dei suggerimenti e dei team.
- **Team e TeamManagement:** La classe TeamManagement funge da associazione tra Admin e Team, gestendo ruoli e stato di attività (*isActive*). Ogni team può avere molteplici assegnazioni (Assignment).
- **ClassUT:** Entità fondamentale che rappresenta la classe oggetto dei test. È legata a suggerimenti (Hint), operazioni di sistema e agli avversari automatizzati (Opponent).
- **Scalata:** Rappresenta una sequenza di sfide. È legata a ClassUT tramite una relazione di *aggregazione*, indicando che una scalata è composta da una o più classi da testare.
- **Hint:** Gestito dall'amministratore, fornisce supporto agli utenti per specifiche ClassUT.
- **Interaction e Operation:** Classi dedicate al logging delle azioni degli utenti e delle operazioni amministrative, mantenendo un riferimento all'admin e alla classe coinvolta.

- **Achievements:** Entità dedicata alla persistenza degli obiettivi sbloccabili dagli utenti in base alle statistiche e al progresso.
- **Opponent:** Entità principale che definisce l'avversario. Utilizza una relazione di *composizione* verso le classi di punteggio, indicando che queste ultime sono parti integranti dell'avversario.
- **EvosuiteScore e JacocoScore:** Classi specifiche per i risultati dei rispettivi tool. Entrambe sono composte da istanze della classe Coverage.
- **Coverage:** Classe granulare che memorizza i dati atomici di copertura: elementi coperti (*covered*) e mancanti (*missed*).

In conclusione, i componenti del microservizio T1 che saranno impattati nelle modifiche sono:

- **Model:** tutte le classi che rappresentano le entità del dominio;
- **Repository:** conversione di repository basati su Mongo in repository basati su SQL.
- **Service:** adattamento della logica di business in seguito alle modifiche effettuate su entità e repository.

Si noti che l'impatto relativo ai *repository* non è forte, in quanto sarà sufficiente modificare l'interfaccia estesa da `MongoRepository` a `JpaRepository`, come mostrato di seguito.

```
public interface AdminRepository extends MongoRepository<Admin, String> {
```

Figure 6: Admin Repository MongoDB

```
@Repository 13 usages Rosa Palmentieri +1
public interface AdminRepository extends JpaRepository<AdminEntity, String> {
```

Figure 7: Admin Repository JPA

2.3.2 Gestione Suggerimenti

I suggerimenti vengono rappresentati nel modo seguente sul microservizio.

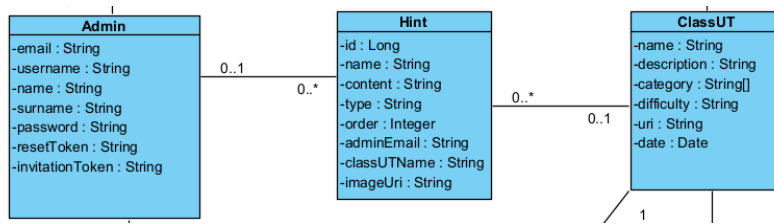


Figure 8: Diagramma delle Classi con Focus su Suggerimenti

Dunque, la gestione dei suggerimenti comporterà delle modifiche sui seguenti componenti:

- **Model:** aggiunta della nuova entità suggerimento;
- **Repository:** aggiunta del nuovo repository per la persistenza dei dati relativi alla nuova entità;
- **Service:** aggiunta di un nuovo service per implementare la logica di business relativa alla gestione dei suggerimenti;
- **Controller:** aggiunta di un nuovo controller per la definizione ed esposizione delle API di gestione dei suggerimenti;
- **View:** aggiunta di una nuova view per la visualizzazione e gestione dei suggerimenti lato interfaccia utente.

2.4 Diagramma dei Casi d'Uso

Di seguito è riportato un diagramma dei casi d'uso relativo alla funzionalità di gestione dei suggerimenti.

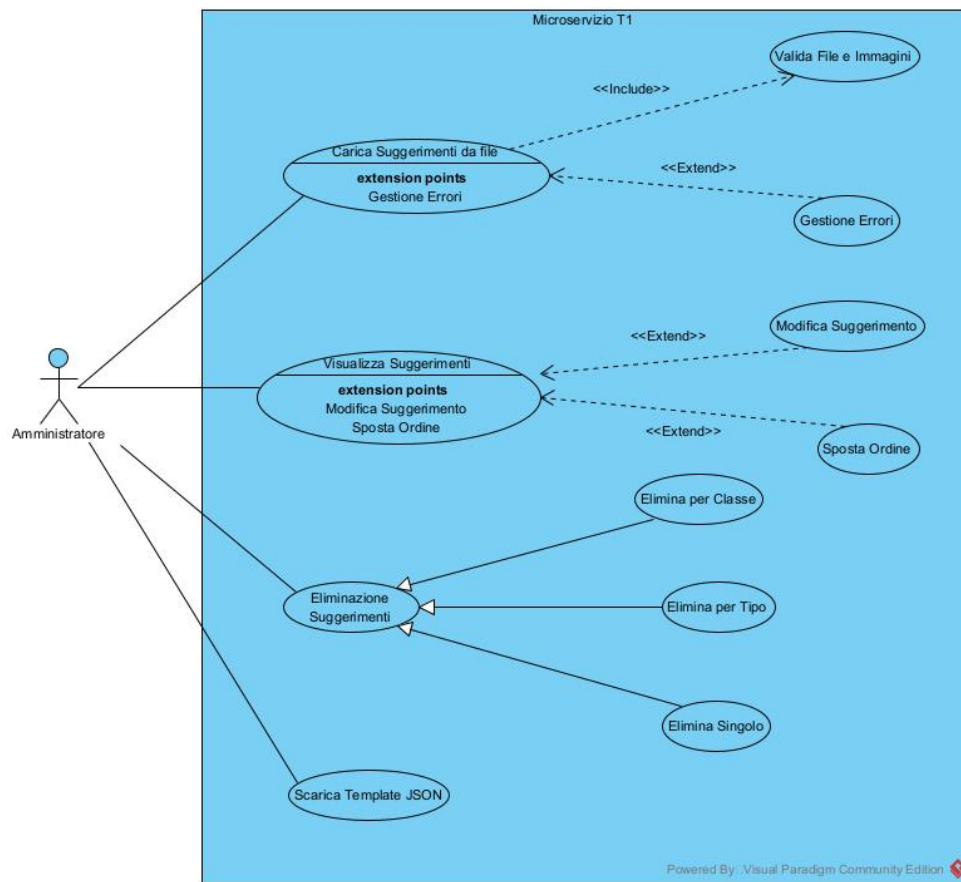


Figure 9: Diagramma dei casi d'uso

Il diagramma dei casi d'uso descrive le interazioni tra l'attore **Admin** e il sistema di gestione dei *Hint*.

I casi d'uso principali sono:

- **Caricamento Suggestimenti da file:** L'amministratore può caricare un file JSON contenente più suggerimenti insieme alle relative immagini.
 - Questo caso d'uso include obbligatoriamente la **Validazione dei dati**, che verifica il formato del file (.json), la validità delle immagini (.png, .jpg) e l'esistenza delle classi UT nel database.
- **Visualizza Suggestimenti:** Una volta visualizzati i suggerimenti tramite filtri dinamici, l'Admin può:

- **Aggiornare:** Modifica di nome, contenuto o file immagine di un singolo suggerimento.
- **Riordinare:** è possibile alterare l'ordine di visualizzazione (UP/-DOWN) per ogni classe.
- **Eliminazione Suggerimenti:** Il sistema supporta tre livelli di eliminazione:
 - *Massiva per Classe:* Rimuove tutti i suggerimenti di una specifica classe UT.
 - *Massiva per Tipo:* Rimuove tutti i suggerimenti Generici o relativi alle Classi.
 - *Puntuale:* Rimuove un singolo suggerimento identificato dalla combinazione di classe e posizione (order).
- **Scarica Template JSON:** Download del template JSON predefinito per guidare l'Admin nella creazione corretta dei file di upload.

Tutti i casi d'uso (eccetto il download del template) richiedono la verifica del **JWT Token** per garantire che l'attore sia un amministratore autorizzato.

2.5 Scenari

ID e Nome	UC1 – Caricamento suggerimenti
Scenario Principale	1. L'Amministratore seleziona “Carica suggerimenti”. 2. Carica un file JSON e le relative immagini. 3. Il sistema verifica la validità del token JWT. 4. Il sistema valida il formato dei file e la struttura del JSON. 5. Il sistema verifica che le classi (<code>ClassUT</code>) referenziate esistano nel database. 6. Il sistema salva le immagini sul server e i dati nel database, assegnando l'ordine progressivo. 7. Messaggio di successo.
Estensioni	• UC1-E1: <code>ClassUT</code> non trovata → Errore 400. • UC1-E2: Immagine non supportata o mancante rispetto al JSON. • UC1-E3: Suggerimento già esistente (duplicato).

Table 12: Caso d'Uso UC1 – Caricamento suggerimenti

ID e Nome	UC2 – Visualizza suggerimenti
Attori	Amministratore, Servizio T5
Scenario	1. L'amministratore accede alla sezione "Suggerimenti". 2. Seleziona la voce "Suggerimenti Generici". 3. Visualizza l'elenco dei suggerimenti generici. 4. Seleziona un suggerimento generico. 5. Visualizza il dettaglio del suggerimento selezionato.

Table 13: Caso d'Uso UC2: Visualizza Suggerimenti

ID e Nome	UC2 – Modifica suggerimenti
Attore	Amministratore
Precondizioni	Login effettuato.
Scenario	1. Accesso sezione "Suggerimenti". 2. Selezione di uno tipo di suggerimento. 3. Selezione di uno specifico suggerimento. 4. Visualizzazione dettaglio. 5. Click sul bottone modifica. 6. Modifica dei dati. 7. Salvataggio modifiche. 8. Messaggio di successo.
Estensioni	• File/Dati invalidi → Messaggio di errore.

Table 14: Caso d'Uso UC2: Modifica Suggerimenti

ID e Nome	UC3 – Eliminazione suggerimenti
Scenario	1. L'Amministratore sceglie la modalità di eliminazione: • Per intera classe UT. • Per tipologia (Generic/Class). • Per singolo suggerimento (Classe + Ordine). 2. Il sistema identifica gli hint e rimuove i file immagine associati dal file system. 3. Il sistema cancella i record dal database.

Table 15: Caso d'Uso UC3 – Eliminazione suggerimenti

ID e Nome	UC4 – Sposta ordine suggerimento
Attore	Amministratore
Scenario	1. L'Admin visualizza la lista degli hint per una classe. 2. Seleziona un suggerimento e clicca su "Sposta Su" o "Sposta Giù". 3. Il sistema individua il suggerimento adiacente. 4. Il sistema inverte i valori del campo order tra i due record. 5. La lista viene aggiornata visivamente.

Table 16: Caso d'Uso UC4 – Sposta ordine suggerimento

ID e Nome	UC5 – Download Template JSON
Scenario	1. L'Admin clicca su "Scarica Template". 2. Il sistema genera un file JSON con la struttura corretta (GENERIC e CLASS). 3. Il file viene scaricato nel browser.

Table 17: Caso d'Uso UC5 – Download Template JSON

3 Progettazione della Soluzione

3.1 Decisioni di Progetto

Per la migrazione verso un database relazionale, la scelta del database è ricaduta su **PostgreSQL** per i seguenti motivi:

- Si tratta di un database relazionale orientato agli oggetti (ORDBMS), pertanto risulta più semplice l'interfacciamento con il back-end, orientato agli oggetti.
- L'insieme dei tipi di dati è esteso, includendo un supporto JSON più avanzato.
- Presenta un sistema di vincoli tra le tabelle più robusto, garantendo l'integrità dei dati e le proprietà ACID per le transazioni.
- È considerato il database più avanzato, robusto e conforme a livello tecnico, ideale per carichi di lavoro complessi e integrità dei dati.
- La natura open source del software ha permesso l'utilizzo senza costi di licenza, mantenendo l'architettura allineata con gli standard dei progetti accademici e open source.

La scelta è stata effettuata anche ragionando in un'ottica di robustezza più a lungo termine. Il progetto in questione è in continua evoluzione, sia in termini di funzionalità aggiuntive sia in termini di modifica delle funzionalità esistenti. PostgreSQL anche se richiede un utilizzo più elevato di memoria e presenta una velocità di lettura leggermente più lenta rispetto ad altri database, consente di gestire carichi di lavoro più complessi e garantisce un'integrità dei dati più robusta.

Un'altra decisione importante presa in riferimento alla migrazione del database riguarda il refactoring del codice preesistente. La migrazione comporterà la modifica dei *model* e dei *repository*. Tali modifiche impatteranno inevitabilmente anche i *service*. Nella struttura di partenza del microservizio i *service* sono stati gestiti direttamente come delle classi. Ma in riferimento ad uno dei principi fondamentali dell'ingegneria del software, **Information Hiding**, riteniamo sia più opportuno riscrivere tali *service* definendo prima l'interfaccia e poi la classe che la implementa.

Un'altra decisione presa riguardo al database è stata quella di adottare il modulo **Flyway**, che permette di definire tutte le modifiche allo schema del database (come creazione di tabelle, aggiunta di colonne, modifica di indici e così via) come file di script SQL separati, con lo scopo di versionare il database.

Ogni script è contrassegnato con un numero di versione, per esempio V1.0.1__create_admin_table.sql, e Flyway esegue sempre gli script in ordine numerico, garantendo che le modifiche vengano applicate nello stesso modo, in

ogni ambiente (sviluppo, test, produzione).

Inoltre, viene creata la tabella *flyway_schema_history* all'interno del database, che registra quali migrazioni sono state applicate e quando. Quando l'applicazione si avvia, Flyway controlla questa tabella, vede quali migrazioni mancano e applica solo quelle, senza mai rieseguire quelle già completate. Questo rende il processo ripetibile e non distruttivo.

In riferimento alla gestione dei suggerimenti, si è deciso di estendere il modello dati preesistente con una nuova tabella: *hint*. Il suggerimento potrà essere di due tipi:

- generico
- associato ad una classe UT

Sarà possibile associare eventualmente un'immagine ad un suggerimento. Inoltre, deve essere definito l'ordine di visualizzazione dei suggerimenti, fondamentale in fase di recupero di questi da parte del microservizio T5.

Per garantire la persistenza delle informazioni relative ai suggerimenti richieste nei requisiti e sulla base delle considerazioni appena fatte, si è deciso di prevedere per tale tabella i campi:

- **name**: nome del suggerimento;
- **content**: contenuto del suggerimento;
- **type**: tipo di suggerimento, *generic* oppure *class* ovvero associato ad una classe UT;
- **class_ut_name**: nome della classe UT se il suggerimento è di tipo associato ad una classe;
- **admin_email**: email dell'amministratore che ha creato il suggerimenti;
- **image_uri**: URI dell'eventuale immagine allegata al suggerimento;
- **order**: ordine di visualizzazione del suggerimento.

Per il caricamento dei suggerimenti si è deciso di adottare un file di tipo JSON perchè questo formato offre la possibilità di visualizzare il suggerimento in maniera strutturata con le informazioni di nostro interesse. Inoltre, il mapping tra un file JSON ed un oggetto Java risulta anche più semplice rispetto ad altri tipi di file come .txt o .html.

3.2.1 Component Diagram

Il diagramma dei componenti mostra la struttura statica dell'intera applicazione, con l'aggiunta delle nuove API esposte dal modulo T1.

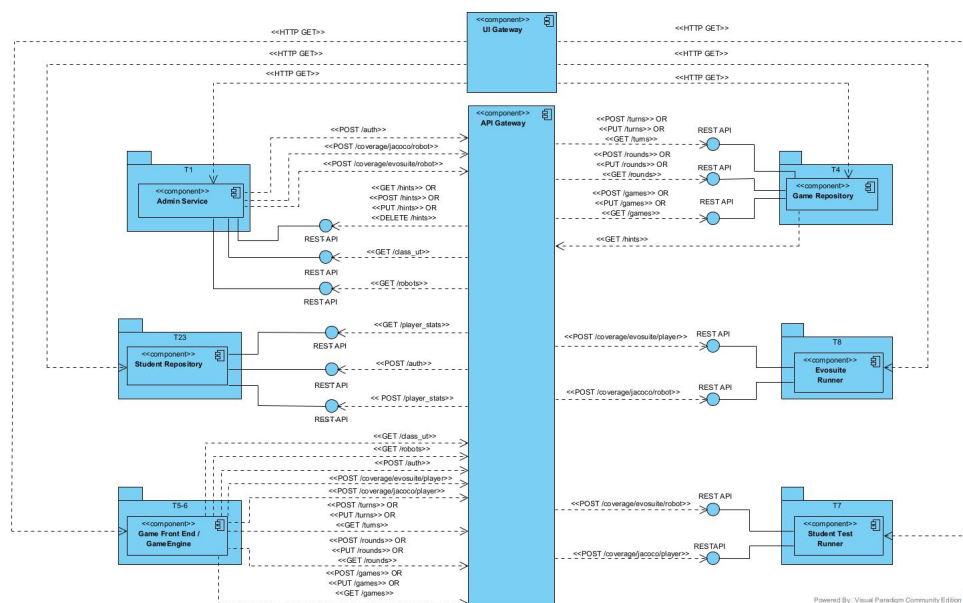


Figure 10: Component Diagram

- **T1: Admin Service:** espone diverse interfacce *REST API*, tra cui la nuova gestione `/hints` con i metodi GET, POST, PUT, DELETE. Inoltre, fornisce endpoint per la gestione di `/class-ut` e `/robots`.
- **UI Gateway:** agisce come punto di ingresso per il traffico proveniente dal Web Browser dell'amministratore. Comunica direttamente con T1 tramite chiamate HTTP GET/POST per il caricamento della dashboard e delle risorse statiche.
- **API Gateway:** costituisce il bus di comunicazione centrale del sistema. T1 utilizza questo componente come mediatore per accedere ai servizi interni, garantendo il disaccoppiamento tra i microservizi.
- **T23: Student Repository:** T1 interagisce con questo componente attraverso l'API Gateway per la gestione dei dati relativi agli studenti e per le operazioni di autenticazione (`/auth`) e recupero statistiche (`/player-stats`).

- **T4: Game Repository:** interagisce con T1 attraverso l'API Gateway per la visualizzazione dei suggerimenti.

3.2.2 Package Diagram

Il diagramma dei package mostra l'organizzazione modulare del codice sorgente del microservizio T1 (`manvsclass`), evidenziando la decomposizione del sistema in package e le relative dipendenze architetturali. La struttura segue il pattern a strati (Layered Architecture) tipico delle applicazioni Spring Boot.

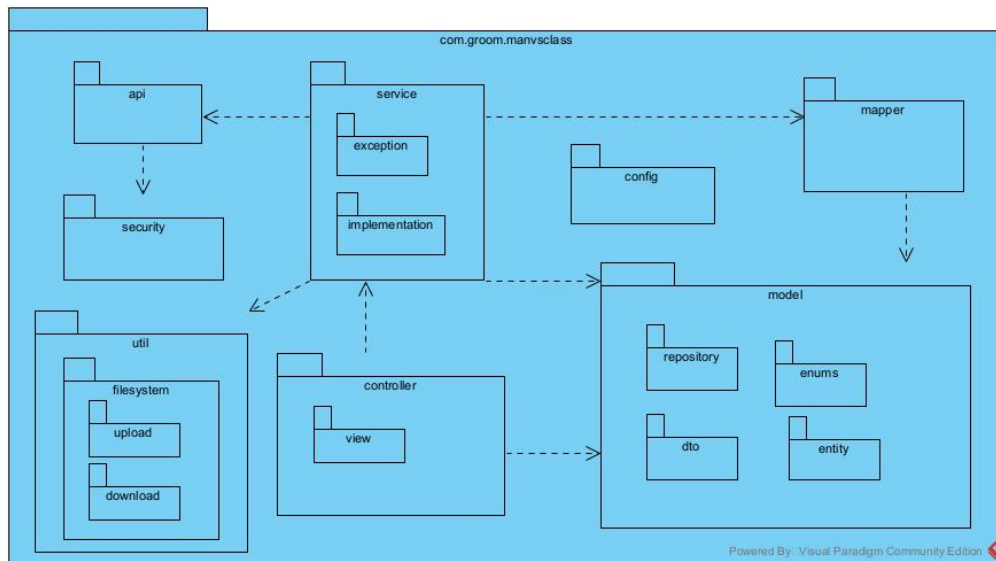


Figure 11: Diagramma dei Package

Il sistema è organizzato sotto il namespace radice `com.groom.manvsclass` e suddiviso nei seguenti strati funzionali:

- **Controller Layer:** Gestisce l'interfaccia verso l'esterno, esponendo le API REST. Le classi qui contenute ricevono le richieste HTTP e delegano l'elaborazione al livello di servizio. Dipende direttamente da `service` per la logica e da `model` per i dati.
- **Service Layer:** Incapsula la logica di business. Definisce le interfacce dei servizi nel package principale.
 - **service.implementation:** Contiene le implementazioni concrete della logica di business. Dipende dal layer `repository` per la persistenza.

- **Data Access Layer:** Si tratta del `repository`, che fornisce l'astrazione per l'accesso ai dati (Spring Data JPA) interagendo con il database. Dipende da `model.entity`.
- **Domain Model:** Pacchetto trasversale contenente le strutture dati condivise. È suddiviso in:
 - **entity:** Classi mappate sul database (JPA Entities).
 - **dto:** Oggetti per il trasferimento dati (Data Transfer Objects).
 - **enums:** Enumerazioni di dominio.
 - **mapper:** Componenti per la conversione Entity-DTO.
- **Configuration:** Contiene le classi di configurazione del framework Spring e dell'applicazione.

3.2.3 Sequence Diagram

Il diagramma di sequenza mostrato di seguito mostra il flusso di interazioni per il caso d'uso “**Creazione Suggerimenti**” tramite caricamento di file JSON e relative immagini.

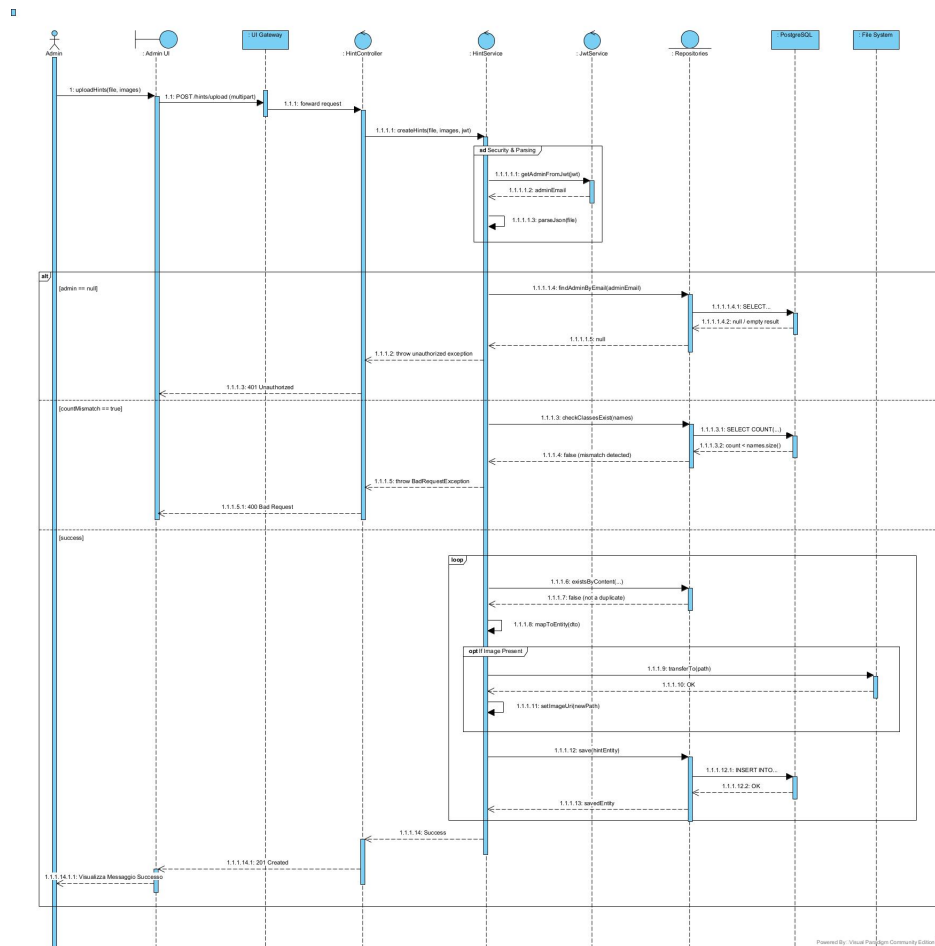


Figure 12: Sequence Diagram Creazione Suggerimenti

Il processo è avviato dall'attore **Admin** e attraversa i seguenti layer e logiche condizionali:

1. **Inoltro e Parsing della Richiesta:** La richiesta POST multipart, contenente il file JSON e le immagini, viene inoltrata dal **:UI Gateway** al **:HintController**. Quest'ultimo delega l'elaborazione al **:HintService**. Il servizio esegue due operazioni preliminari:

- Interroga il **:JwtService** per estrarre l'identificativo dell'amministratore dal token di sicurezza.
- Effettua il parsing del file JSON (auto-chiamata `parseJson`) per convertirlo in una lista di oggetti DTO.

2. **Validazioni Bloccanti (Frammento `alt`)**: Prima di procedere, il sistema esegue validazioni critiche racchiuse in un frammento combinato di tipo *alternative*. Solo uno dei seguenti percorsi viene eseguito:
- **Guardia [`admin == null`]**: Se l'amministratore non viene trovato nel database, il flusso si interrompe immediatamente restituendo un errore 401 `Unauthorized` all'interfaccia utente.
 - **Guardia [`else if countMismatch == true`]**: Se una o più classi (`ClassUT`) specificate nel JSON non esistono nel sistema, il flusso si interrompe restituendo un errore 400 `Bad Request`.
 - **Guardia [`else`]** (Percorso di Successo): Se entrambe le validazioni precedenti vengono superate, il flusso prosegue nel blocco principale di elaborazione.
3. **Elaborazione Iterativa (Frammento `loop`)**: All'interno del percorso di successo, un frammento *loop* itera su ogni DTO di suggerimento valido:
- Viene verificata l'assenza di duplicati tramite il repository.
 - Il DTO viene mappato in una entità di persistenza.
 - **Gestione Immagine (Frammento `opt`)**: Se per il suggerimento corrente è presente un file immagine, questo viene salvato fisicamente sul `:File System` e il percorso viene aggiornato nell'entità.
 - Infine, l'entità completa viene persistita sul database `:PostgreSQL`.

Al termine del ciclo, il servizio notifica il successo al controller, che restituisce un codice HTTP 201 `Created` all'Admin UI, confermando l'avvenuta operazione.

Il diagramma di sequenza seguente mostra il flusso di “**Visualizzazione Suggerimenti**”.

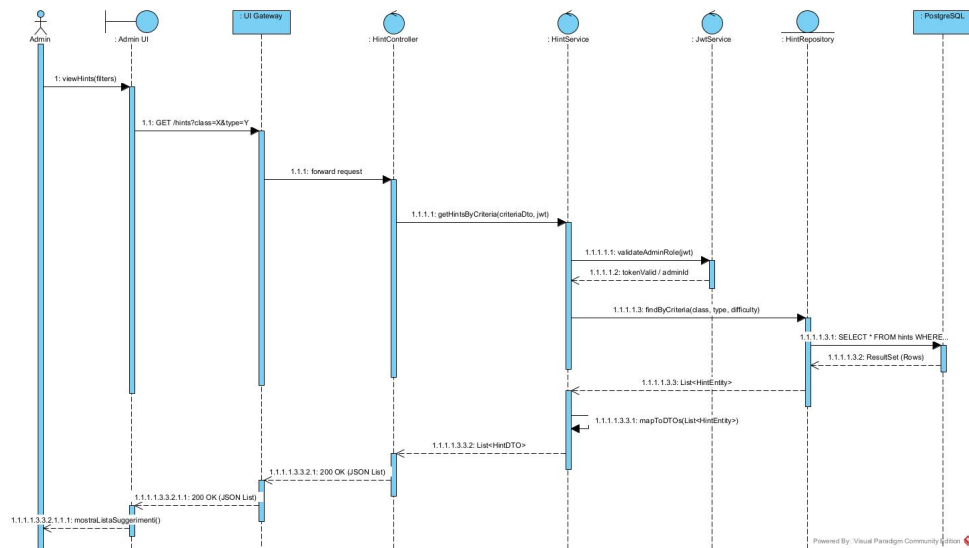


Figure 13: Sequence Diagram Visualizzazione Suggerimenti

Il processo segue un flusso di lettura standard attraverso i layer architetturali:

1. **Inoltro della Richiesta e Sicurezza:** L'attore **Admin** richiede la visualizzazione tramite l':**Admin UI**, specificando eventuali filtri di ricerca (es. per classe o difficoltà). La richiesta GET viene inoltrata dal **:UI Gateway** al **:HintController**. Il controller delega al **:HintService**, il quale, come primo passo, interpella il **:JwtService** per validare il token di sessione e assicurarsi che il richiedente abbia i permessi di amministratore necessari per la lettura.
2. **Accesso ai Dati:** Una volta validata la richiesta, il **:HintService** invoca il **:HintRepository**. Quest'ultimo traduce i criteri di filtro in una query SQL (SELECT) eseguita sul database fisico **:PostgreSQL**. Il database restituisce le righe trovate, che il repository converte in una lista di oggetti **HintEntity** e restituisce al servizio.
3. **Trasformazione e Risposta:** Il **:HintService** esegue una trasformazione fondamentale (auto-chiamata `mapToDTOs`), convertendo la lista di entità interne del database in una lista di Data Transfer Objects (DTO), disaccoppiando così il modello di persistenza dall'API esterna. La lista di DTO viene restituita al controller, che la serializza in formato JSON e la invia indietro attraverso il gateway fino all'interfaccia utente per la visualizzazione finale.

Il diagramma di sequenza seguente mostra il flusso “**Modifica Suggerimento**”.

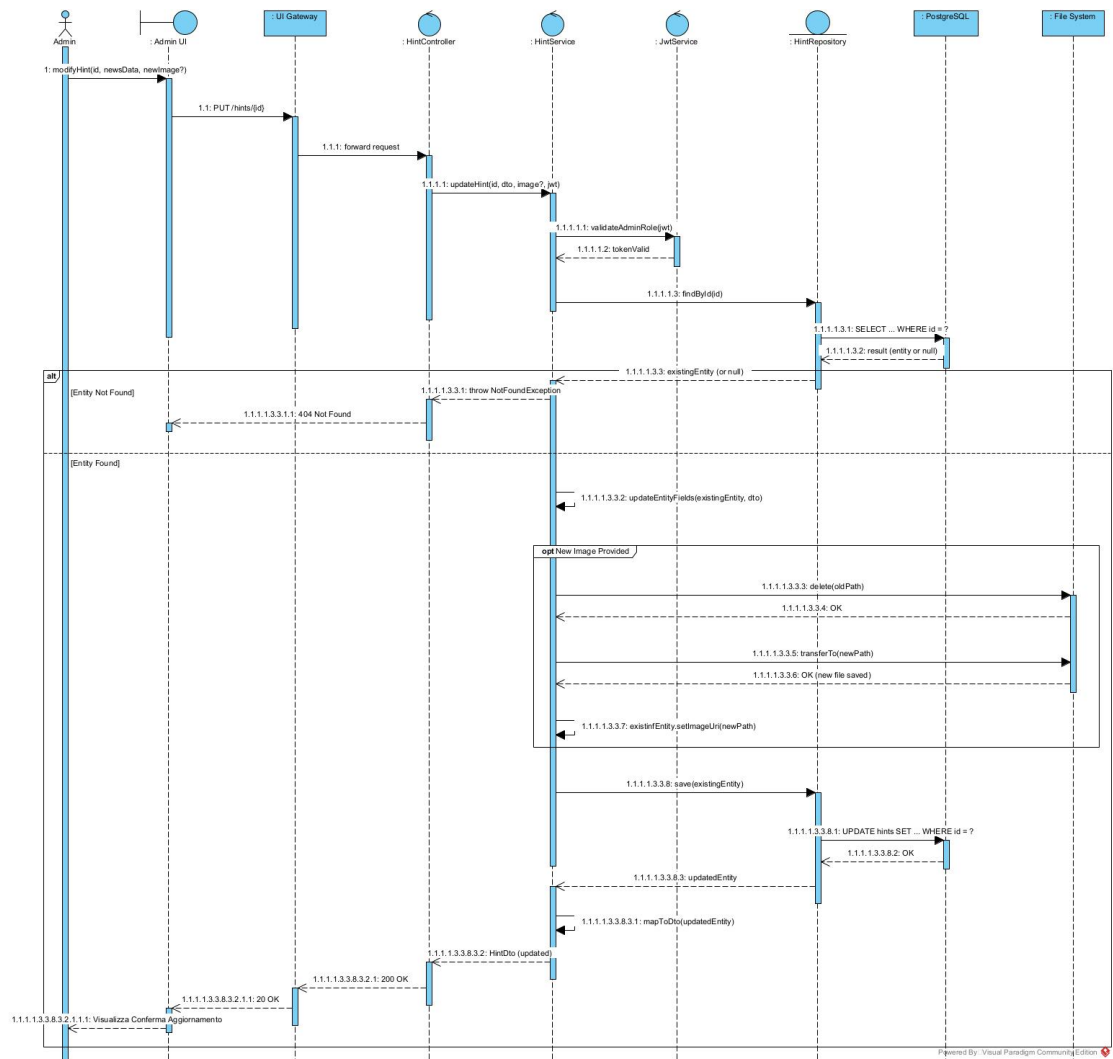


Figure 14: Sequence Diagram Aggiornamento Suggestimenti

Il processo è avviato dall'amministratore e segue un percorso che garantisce l'integrità dei dati e delle risorse:

1. **Ricezione e Validazione della Richiesta:** Una richiesta PUT viene inoltrata attraverso i layer **:UI Gateway** e **:HintController** fino al **:HintService**. Come da prassi, il servizio valida preliminarmente il token di sicurezza tramite il **:JwtService**.
2. **Recupero Condizionale (Frammento **alt**):** Prima di applicare qualsiasi modifica, è cruciale verificare l'esistenza dell'entità target. Il servizio

interroga il **:HintRepository** (e quindi **:PostgreSQL**) tramite `findById(id)`. L'esito determina il flusso successivo tramite un frammento *alternative*:

- **Guardia [Not Found]**: Se l'ID non corrisponde a nessun suggerimento, il flusso si interrompe restituendo un errore 404 Not Found all'utente.
- **Guardia [Found]**: Se l'entità esiste, il flusso procede nel blocco principale di aggiornamento.

3. **Logica di Aggiornamento e Gestione Risorse (Frammento opt)**:

All'interno del blocco di successo, il servizio aggiorna i campi dell'entità recuperata con i nuovi dati. Un frammento *optional* gestisce la logica specifica per le immagini:

- **Guardia [New Image Provided]**: Se la richiesta include una nuova immagine, il servizio interagisce con il **:File System** per eliminare fisicamente il file precedente e salvare quello nuovo, aggiornando di conseguenza il riferimento nell'entità.

4. **Persistenza delle Modifiche**: Lo stato aggiornato dell'entità viene reso persistente invocando il metodo `save` del repository, che si traduce in un comando SQL UPDATE sul database. Infine, l'entità aggiornata viene convertita in DTO e restituita lungo la catena fino all'interfaccia utente (200 OK).

Il diagramma di sequenza seguente mostra il flusso per l'operazione di “**Eliminazione Suggerimenti**”.

1. **Inoltro e Validazione della Richiesta:** Una richiesta DELETE, specificando i parametri identificativi (`classUT` e `order`), viene inoltrata attraverso i layer **:UI Gateway** e **:HintController** fino al **:HintService**. Il servizio valida il token di sicurezza tramite il **:JwtService**.
2. **Ricerca Condizionale (Frammento alt):** Il servizio deve identificare univocamente il suggerimento da eliminare. La logica di query varia in base al tipo di suggerimento, gestita da un frammento *alternative*:
 - **Guardia [`classUT` == "null"]:** Se il parametro della classe è nullo, si tratta di un suggerimento "GENERIC". Il repository viene interrogato per tipo e ordine.
 - **Guardia [`classUT` != "null"]:** Se è specificata una classe, si tratta di un suggerimento di tipo "CLASS". Il repository viene interrogato per nome della classe e ordine.
3. **Gestione Eliminazione e Pulizia (Frammento alt):** L'esito della ricerca determina il flusso successivo:
 - **Guardia [`Hint Non Trovato`]:** Se nessun suggerimento corrisponde ai criteri, il flusso si interrompe con un errore 404 Not Found.
 - **Guardia [`Hint Trovato`]:** Se l'entità viene trovata, si procede con la cancellazione fisica.
4. **Cancellazione Fisica:**
 - **Database:** Il servizio invoca il metodo `delete` del repository, che esegue il comando SQL DELETE sul **:PostgreSQL**.
 - **File System (Frammento opt):** Un frammento *optional* verifica se al suggerimento è associata un'immagine ([Se `ImageUri` esiste]). In caso affermativo, il servizio richiede al **:File System** la rimozione fisica del file.

Infine, una conferma di successo (200 OK) viene restituita all'interfaccia utente.

3.2.4 Deploy Diagram

Il diagramma di deployment mostra l'architettura fisica del sistema in un ambiente di produzione, evidenziando la distribuzione dei componenti software (artefatti) sui nodi computazionali e i protocolli di comunicazione utilizzati.

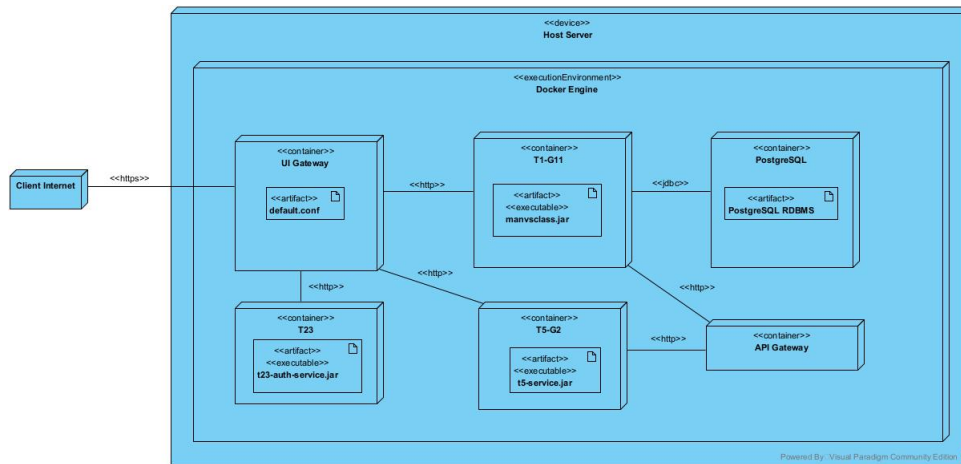


Figure 16: Deploy Diagram

L'architettura è basata su container Docker. L'intera infrastruttura è ospitata su un nodo fisico principale **Host Server** (stereotipo `<<device>>`). Al suo interno è in esecuzione il **Docker Engine** (`<<executionEnvironment>>`), che funge da runtime per i container isolati.

Ogni microservizio e componente infrastrutturale è incapsulato nel proprio nodo di esecuzione (`<<container>>`):

- **UI Gateway:** Punto di ingresso unico per il traffico esterno. Esegue il server web Nginx, configurato tramite l'artefatto `default.conf`.
- **T1-G11:** Ospita il servizio di backend principale per l'amministrazione, eseguendo l'artefatto Java `manvsclass.jar`.
- **T23:** Dedicato al servizio di autenticazione, esegue l'artefatto `t23-auth-service.jar`.
- **T5:** Contiene la logica del microservizio T5 (artefatto `t5-service.jar`).
- **API Gateway:** Agisce come router per specifiche comunicazioni interne tra microservizi (artefatto `api-gateway.jar`).
- **PostgreSQL:** Gestisce la persistenza dei dati relazionali (artefatto DBMS Data).

Le comunicazioni di rete sono definite attraverso percorsi specifici con protocolli e porte dedicate:

1. **Accesso Esterno:** Il traffico proveniente dal *Client Internet* raggiunge il **UI Gateway** tramite protocollo sicuro `<<https>>` sulla porta 443.

2. **Routing Diretto (Nginx):** Come definito dalla configurazione del reverse proxy, il UI Gateway instrada le richieste direttamente ai container di competenza tramite la rete interna Docker (protocollo `«http»`):
 - Verso **T1-G11** sulla porta 8081 (per funzionalità amministrative).
 - Verso **T23** sulla porta 8082 (per login e gestione profilo).
 - Verso **T5** sulla porta 8085 (per funzionalità di gioco).
3. **Comunicazione Inter-Service (Backend-to-Backend):** Il servizio **T5** necessita di comunicare con T1. Questo flusso non è diretto ma mediato: T5 invia richieste `«http»` all'**API Gateway** (porta 8090), che a sua volta le inoltra a **T1-G11** (porta 8081).
4. **Persistenza Dati:** Il container **T1-G11** comunica con il **PostgreSQL Container** utilizzando il protocollo standard `«jdbc»` sulla porta 5432.

3.2.5 Nuove Interfacce REST

Di seguito vengono descritte le interfacce REST esposte dal microservizio T1.

Nome	Upload Massivo Suggerimenti
Metodo	POST
Endpoint	/hints/upload
Headers	Cookie: jwt
Body	• file: File JSON (array suggerimenti). • images: Lista immagini (opzionale).
Response	• 200 OK: Suggerimenti caricati con successo. • 400 Bad Request: JSON malformato, campi vuoti o mancanti, estensione file errata. • 404 Not Found: Classe UT associata non trovata. • 409 Conflict: Suggerimento già esistente (contenuto duplicato). • 415 Unsupported Media Type: Formato file non supportato (non JSON o immagini non valide). • 401 Unauthorized: Token mancante o non valido.

Table 18: API Upload Suggerimenti

Nome	Visualizzazione Suggerimenti
Metodo	GET
Endpoint	/hints
Params	• classUTName, type, order.
Response	• 200 OK: Lista suggerimenti (o lista vuota se nessun risultato trovato). • 400 Bad Request: Parametri di ricerca non validi. • 401 Unauthorized: Token mancante o non valido.

Table 19: API Visualizzazione Suggerimenti

Nome	Eliminazione Suggerimenti
Metodo	DELETE
Endpoints	• /hints/{classUTName} (Per classe) • /hints/className/{class}/order/{n} (Singolo) • /hints/type/{type} (Tutti i generici)
Response	• 200 OK: Eliminazione effettuata con successo. • 404 Not Found: Risorsa da eliminare non trovata. • 401 Unauthorized: Non autorizzato.

Table 20: API Eliminazione Suggerimenti

Nome	Modifica e Ordinamento
Update	PUT /hints/update/{id} <i>Body:</i> Multipart (name, content, file)
Move	POST /hints/{id}/move <i>Params:</i> direction (UP/DOWN)
Response	• 200 OK: Modifica o spostamento avvenuti con successo. • 400 Bad Request: Dati di input non validi o mancanti. • 404 Not Found: ID suggerimento non trovato. • 409 Conflict: La modifica crea un duplicato di un suggerimento esistente. • 401 Unauthorized: Non autorizzato.

Table 21: API Modifica e Ordinamento

4 Implementazione

4.1 Tabella Moduli Aggiunti e Modificati

Service	Tipo di Modifica
T1	Migrazione database da MongoDB a PostgreSQL
T1	Modifica database per aggiungere la tabella <i>hints</i> dei suggerimenti
T1	Tutte le entity sono state spostate sotto il package /model/entity
T1	Tutte le entity preesistenti sono state modificate per essere adattate al database sottostante
T1	Tutti i repository sono stati modificati per estendere l'interfaccia JpaRepository
T1	Tutti i service preesistenti sono stati modificati e trasformati in interfacce
T1	Per ciascuna interfaccia service è stata creata una classe di implementazione (Impl), all'interno del package /service/implementation
T1	Tutti i controller preesistenti sono stati modificati per riadattarli a entity e service ridefiniti
T1	È stata creata la nuova entity HintEntity
T1	È stato creato il nuovo repository HintRepository
T1	Sono stati creati i DTO Hint, HintResponse e HintUpdatedto
T1	È stato creato un enum HintTypeEnum
T1	È stato aggiunto il package mapper
T1	È stato creato il mapper HintMapper per convertire entity in DTO e viceversa
T1	È stata creata l' interfaccia HintService
T1	È stata creata la classe HintServiceImpl che implementa l'interfaccia HintService
T1	È stato creato il controller HintController
T1	È stata creata la classe di utility HintSpecification per filtrare i dati sul database
T1	È stata creata la classe di configurazione JpaConfig per la configurazione del database Postgres
T1	È stata creata la classe di configurazione WebConfig
T1	È stata creata la classe di configurazione WebSecurityConfig
T1	È stata creata la classe di configurazione FlywayConfig per la configurazione di Flyway
T1	È stata creata la view HintViewController
T1	Sono stati modificati i file dashboard.html, dashboard.js per aggiungere la card relativa ai suggerimenti
T1	È stata aggiunta la directory resources/static/t1/js/hint per raccogliere i file javascript relativi ai suggerimenti
T1	Sono stati creati i file hint_main.js, hint_upload.js per gestire lato frontend le logiche di visualizzazione, inserimento ed eliminazione

Service	Tipo di Modifica
T1	È stata aggiunta la directory <code>resources/templates/hint</code> per raccogliere i file html relativi ai suggerimenti
T1	Sono stati creati i file <code>hint_main.html</code> , <code>hint_upload.html</code>
T1	Sono stati modificati i file <code>message.properties</code> , <code>message.en.properties</code> e <code>message.it.properties</code> per i messaggi internazionalizzati
T1	È stato modificato il file <code>application.properties</code> per aggiungere le properties relative al nuovo database
T1	È stato aggiornato il file <code>.env</code> con le nuove variabili d'ambiente
T1	È stato aggiornato il file <code>pom.xml</code> per aggiungere le dipendenze relative a Postgres, MapStruct, Flyway
UI Gateway	È stato aggiornato il file <code>T1-route.conf</code> per aggiungere le rotte relative ai suggerimenti

Table 22: Elenco delle principali modifiche ai microservizi

5 Test

Di seguito è riportata la tabella dei test eseguiti per la funzionalità di **Creazione Suggerimenti**.

Nome	Test	Risultato Atteso	Esito
Caricamento File JSON	HintsWithImages.json	Suggerimenti caricati con successo.	PASS
Caricamento file non JSON	FileNonJson.txt	Formato file non valido. Assicuratevi che sia un JSON valido.	PASS
Caricamento file con contenuto errato	StructureWrong.json	Il contenuto del file non è valido.	PASS
Caricamento file senza immagine allegata	HintsWithImages.json	URI dell'immagine trovato nel JSON, ma file non allegato.	PASS
Caricamento dello stesso file due volte	HintsWithImages.json	Suggerimento già esistente (contenuto duplicato).	PASS
Caricamento file vuoto	Empty.json	Il file non contiene suggerimenti validi da caricare.	PASS
Caricamento file con tutti i campi con valore una stringa vuota	HintWithAllFieldButEmpty.json	Il file JSON è malformato o contiene errori di sintassi.	PASS
Caricamento file con immagini allegate come file di testo	HintsFTPFile.json	Il file 'image2.txt' non è un'immagine valida. Sono ammessi solo .png, .jpg, .jpeg.	PASS
Caricamento file suggerimenti associati ad una classe UT non esistente	ClassNotFound.json	Classe UT non trovata.	PASS

Table 23: Test Creazione Suggerimenti

Di seguito è riportata la tabella dei test eseguiti per la funzionalità di **Visualizzazione Suggerimenti**.

Nome	Test	Risultato Atteso	Esito
Visualizza suggerimenti generici	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti Generici	Suggerimenti generici visualizzati.	PASS
Visualizza suggerimenti di classe	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti di Classe	Suggerimenti di classe visualizzati.	PASS
Visualizza dettaglio suggerimento generico con ordine 1	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti Generici e poi sul primo suggerimento in cima alla lista	Dettaglio suggerimento generico visualizzato.	PASS
Visualizza dettaglio di un suggerimento di classe	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti di Classe e poi su una classe specifica e poi su un suggerimento	Dettaglio suggerimento di classe visualizzato.	PASS

Table 24: Test Visualizzazione Suggerimenti

Di seguito è riportata la tabella dei test eseguiti per la funzionalità di **Aggiornamento Suggerimenti**.

Nome	Test	Risultato Atteso	Esito
Modifica suggerimento generico	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti Generici e poi su un suggerimento generico specifico e modificare Nome o Contenuto o Immagine	Suggerimento generico modificato con successo.	PASS
Modifica suggerimento di classe	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti di Classe e poi su una classe specifica e poi su un suggerimento specifico e modificare Nome o Contenuto o Immagine	Suggerimento di classe modificato con successo.	PASS

Table 25: Test Aggiornamento Suggerimenti

Di seguito è riportata la tabella dei test eseguiti per la funzionalità di **Eliminazione Suggerimenti**.

Nome	Test	Risultato Atteso	Esito
Eliminazione di tutti i suggerimenti di una classe UT	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti di Classe e poi su una classe specifica e poi sul bottone Elimina	Suggerimenti associati alla classe specifica eliminati.	PASS
Eliminazione di tutti i suggerimenti generici	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti Generici e poi sul bottone Elimina	Suggerimenti generici eliminati.	PASS
Eliminazione suggerimenti generico di uno specifico ordine	Sulla Dashboard cliccare sulla card Suggerimenti e poi sul bottone Suggerimenti Generici e poi su un suggerimento specifico e poi sul bottone Elimina	Suggerimento generico eliminato.	PASS

Table 26: Test Eliminazione Suggerimenti

6 Sviluppi Futuri

Di seguito sono riportate alcune idee di sviluppi futuri da fare:

- Aggiungere paginazione alla GET dei suggerimenti.
- Sviluppare anche una create manuale dei suggerimenti.
- Aggiungere dei mapper per mappare tutte le entity sui DTO senza dover assegnare a mano i campi con i setter.
- Gestire opportunamente le operazioni transazionali avendo ora un database SQL.
- Dare la possibilità di caricare anche file diversi dal JSON.